



U-Flow: A U-Shaped Normalizing Flow for Anomaly Detection with Unsupervised Threshold

Matías Tailanian^{1,2} · Álvaro Pardo^{1,3} · Pablo Musé²

Received: 18 September 2023 / Accepted: 24 April 2024 / Published online: 31 May 2024
© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2024

Abstract

In this work, we propose a one-class self-supervised method for anomaly segmentation in images that benefits from both a modern machine learning approach and a more classic statistical detection theory. The method consists of four phases. First, features are extracted using a multi-scale image transformer architecture. Then, these features are fed into a U-shaped normalizing flow (NF) that lays the theoretical foundations for the subsequent phases. The third phase computes a pixel-level anomaly map from the NF embedding, and the last phase performs a segmentation based on the *a contrario* framework. This multiple hypothesis testing strategy permits the derivation of robust unsupervised detection thresholds, which are crucial in real-world applications where an operational point is needed. The segmentation results are evaluated using the mean intersection over union metric, and for assessing the generated anomaly maps we report the area under the receiver operating characteristic curve (*AUROC*), as well as the area under the per-region-overlap curve (*AUPRO*). Extensive experimentation in various datasets shows that the proposed approach produces state-of-the-art results for all metrics and all datasets, ranking first in most MVTec-AD categories, with a mean pixel-level *AUROC* of 98.74%. Code and trained models are available at <https://github.com/mtailanian/uflow>.

Keywords Anomaly detection · Anomaly localization · Self-supervised learning · One-class classification · Novelty detection · Normalizing flows · A contrario · Number of false alarms · NFA

1 Introduction

Detecting image anomalies is a long-standing problem that has been studied for decades. In recent years, this problem has received a growing interest from the computer vision community, motivated by applications in various fields, ranging from surveillance and security to even health care. One of the most common applications, and the one that we are especially interested in, is automatizing product quality control

in an industrial environment. In this case, it is often very hard (sometimes even unfeasible) to collect and label a good amount of data representing all kinds of anomalies since these are rare structures by definition. For this reason, the major effort on anomaly detection methods has been focused on learning only from the normal samples (i.e., anomaly-free images) and detecting the anomalies as everything that deviates from what has been learned. This approach is found in the literature under different names, usually called one-class classification, self-supervised anomaly, or novelty detection.

There is a wide literature on anomaly detection, and many approaches have been proposed. One of the most common classic approaches consists of embedding the training images in some latent space and then evaluating how far a test image is from the manifold of normal images. Other approaches focus on learning the probability density function of the training set. Among these, three types of generative models have become popular in anomaly detection in the last five years. On the one hand, generative adversarial networks [1, 2] implicitly learn this probability, being able only to sample from it. On the other hand, variational autoencoders [3, 4]

✉ Matías Tailanian
mtailanian@digitalsense.ai

Álvaro Pardo
apardo@digitalsense.ai

Pablo Musé
pmuse@fing.edu.uy

¹ Digital Sense, Brenda 5751, 11400 Montevideo, Uruguay

² Facultad de Ingeniería, Universidad de la República, Av. Julio Herrera y Reissig 565, 11300 Montevideo, Uruguay

³ Departamento de Ingeniería, Universidad Católica del Uruguay, Av. 8 de Octubre 2738, 11600 Montevideo, Uruguay

can explicitly estimate this probability density, but they can only learn an approximation, as they maximize the evidence lower bound. And finally, normalizing flows [5–7] are able to learn the exact probability density function explicitly. Being able to do so has several advantages, as it permits to estimate the likelihood and score of how probable it is that the tested data belongs to the same training distribution. A significant advantage we exploit in this work is the possibility of developing formal statistical tests to derive unsupervised detection thresholds. This is a crucial feature for most real-world problems where a segmentation of the anomaly is needed. To this end, we propose a multiple hypothesis testing strategy based on the *a contrario* framework [8].

Common approaches based on VAEs [9–11] and GANs [2, 12, 13] have demonstrated poor performance in the industrial defect detection case, where anomalies are actually very similar to the normal parts of the images. To enhance the performance in these cases, instead of grounding the detection on the RGB image itself, in recent years, many works have introduced a feature extraction stage using a pre-trained network, usually on ImageNet [14], and perform the anomaly detection in the feature space [15–20].

In this work, we propose **U-Flow**, a new self-supervised method that ensembles the features extracted by multi-scale image transformers into a U-shaped normalizing flow’s architecture and applies the *a contrario* methodology for finding an automatic segmentation of anomalies in images. Our work achieves state-of-the-art results, widely outperforming previous works in terms of *mIoU* for the segmentation, and exhibiting top performance in localization metrics in terms of *AUROC* and *AUPRO*. The main contributions of this paper are:

- For the feature extraction, we propose a multi-scale vision transformer architecture based on the CaiT transformers [21] pre-trained independently for each scale, which we refer to as **MS-CaiT**. Results clearly show that it better captures the multi-scale information and increases performance, comparing it both against other neural networks’ feature extractors and other multi-scale transformers.
- We leverage the well-known advantages of U-like architectures in the setting of normalizing flows, proposing a novel architecture and solving non-trivial technical details for merging the scales in an invertible way that better aggregates the multi-scale information elegantly and efficiently while imposing statistical mutual independence within and across scales.
- We propose, for the first time, to take advantage of the statistical properties of normalizing flows to derive unsupervised thresholds for anomaly detection and segmentation. More precisely, the resulting multi-scale characterization of the anomaly-free images as a white Gaussian process

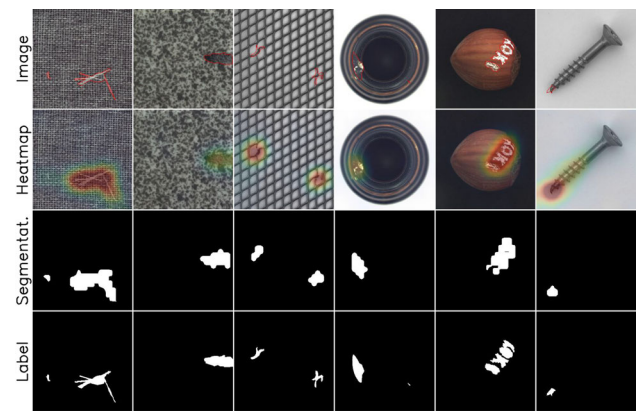


Fig. 1 Anomalies detected with the proposed approach on MVTec-AD examples from different categories. Top row: original images with ground truth segmentation. Second row: corresponding anomaly maps. Third row: automatic segmentations. Last row: ground truth masks

in the latent space allows us to propose a multiple hypothesis testing strategy that leads to an automatic detection threshold to segment anomalies, outperforming state-of-the-art methods.

- Although the method is designed for industrial defect detection tasks, we show that it also exhibits excellent performance on other benchmark datasets of different natures.
- Results are easily reproducible. Code is available on GitHub, and also integrated into *Anomalib* library [22].

In short, we propose an easy-to-train method that does not require any parameter tuning or modification to the architecture (as opposed to other state-of-the-art methods), that is fast, accurate, and provides an unsupervised anomaly segmentation. Example results are shown in Fig. 1.

The remainder of this paper is organized as follows. In Sect. 2, we discuss previous work related to the proposed approach. Details of the method are presented in Sect. 3. In Sect. 4, we compare the performance of our anomaly detection approach with a large number of state-of-the-art methods on several benchmark datasets of different nature: MVTec-AD [23], BeanTech [24], LGG MRI [25], and ShanghaiTech Campus [26]. An ablation study that analyzes the role of each component of the proposed architecture is presented in Sect. 5. We conclude in Sect. 6.

2 Related Work

The literature on anomaly detection is vast, with numerous proposed approaches. In this work, we focus on one-class self-supervised learning for industrial inspection, where a key element is to learn only from anomaly-free images.

Within this group, methods can be further divided into several categories in many different ways.

A large number of methods can be classified as representation-based. These methods proceed by embedding the samples into a latent space endowed with some metric. For example, in CFA [17], the authors propose a way to build features using a patch descriptor and building a scalable memory bank. A special loss function is designed to map features to a hyper-sphere and densely clusterize the representation of normal images. Also, the method can add a few abnormal samples during training that are used to enlarge the distance to the normal samples in the latent space. Similarly, in PatchSVDD [27], the authors construct a hierarchical encoding of image patches and encourage the network to map them to the center of a hyper-sphere in the feature space. This work later inspired [28], where the authors aim to learn more representative features from normal images, using three encoders with three classifiers. In SPADE [29], the anomaly score is obtained by measuring the distance of a test sample to the nearest neighbors (NN) that were computed at the training stage over the anomaly-free images. Both the distance and the NN are calculated in the latent space by constructing a pyramid of features using the most common pre-trained extractors. Additionally, PaDiM [30] proposes to model normality by fitting a multivariate Gaussian distribution after embedding image patches with a pre-trained convolutional neural network (CNN) and base the detection on the Mahalanobis distance to normality. Then, PatchCore [31] further improves SPADE and PaDiM by estimating anomalies with a larger nominal context and making it less reliant on image alignment.

By using an encoder-decoder architecture such as in [4], or a teacher-student scheme like [32], and training the network with only anomaly-free images, the network is supposed to reconstruct normal images accurately but fail to reconstruct anomalies, as it has never seen them during training. This sweet spot when only normal structures can be generated is usually difficult to obtain, and methods tend to generate either bad-quality reconstructions or show a too-good generalization power, causing the anomalies to be well reconstructed too. The basic approach using the reconstruction error as an anomaly score is presented in [9]. Following the same idea, the authors in [10] improve the detection by measuring the reconstruction error with the Structural Similarity index (SSIM), and in [11] the authors limit the reconstruction power by utilizing memory modules in the latent space.

Learning from one-class data leaves some uncertainty on how the method will behave when facing anomalous data. To overcome this potential issue, some works manage to cast the problem into a self-supervised one by creating proxy tasks. For example, in [33], the authors obtain a representation of the images by randomly cutting and pasting regions over anomaly-free images to generate anomalies synthetically.

Regarding the use of normalizing flows (NF) in anomaly detection, a few methods have recently been proposed with impressive results. DifferNet [34] applies an average pooling to the features extracted with a pre-trained network and processes the resulting vectors in a one-dimensional NF, losing important contextual and spatial information. The authors try to alleviate this problem by passing different rotated image versions through the network at the cost of increasing the computational complexity. The method performs only anomaly detection, but a rough localization can also be obtained by back-propagating the negative log-likelihood through the network. CFlow [35] also uses a one-dimensional NF but includes the spatial information using a positional encoding as a conditional vector. On the other hand, Fast-Flow [19] directly uses a two-dimensional NF for each scale independently and averages the results at the end. CS-Flow [20] uses the same NF block as in [34] and [19], but for the affine layer, the information of the different scales are merged, by summing an upsampled/downsampled version of the feature volumes from the other scales. This method extends DifferNet and is also meant only for anomaly detection. However, as all the network's layers are convolutional, the spatial information is preserved, and an anomaly map can be obtained with the forward pass. The results for anomaly detection are excellent, but as the extracted features are of a small spatial size, the performance in localization could be better.

In this work, we further improve the performance of these methods for anomaly localization by proposing a multi-scale transformer-based feature extractor and by equipping the NFs with a fully invertible U-shaped architecture that effectively integrates the information at different scales in a very natural and elegant manner. These two ingredients allow us to embed these multi-scale features in a latent space where intra and inter-scale features are guaranteed to be independent and identically distributed Gaussian random variables. Finally, we design a detection methodology from these statistical properties that leads to statistically meaningful anomaly scores and unsupervised detection thresholds.

3 Methods

The proposed method is depicted in Fig. 2 and consists of four main phases: (1) feature extraction, (2) U-shaped normalizing flow, (3) anomaly score map, and (4) *A contrario* anomaly segmentation. All phases are presented below.

3.1 Phase 1: Feature Extraction

Since anomalies can emerge in various sizes and forms, collecting image information at multiple scales is essential. To do so, the standard deep learning strategy is to use pre-trained

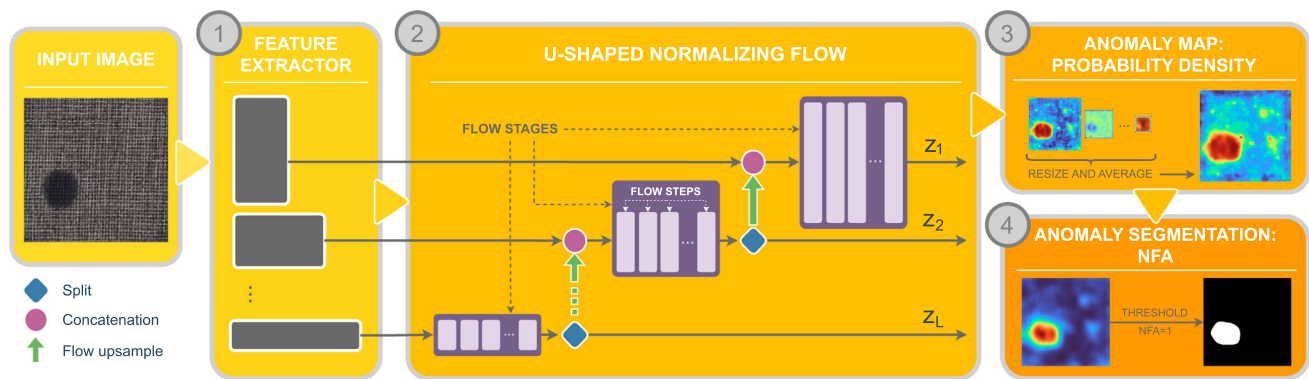


Fig. 2 The method consists of four phases. (1) Multi-scale feature extraction: a rich multi-scale representation is obtained with MS-CaIT by combining pre-trained image transformers acting at different image scales. (2) U-shaped normalizing flow: by adapting the widely used U-like architecture to NFs, a fully invertible architecture is designed. This architecture is capable of merging the information from different scales while ensuring independence in both intra- and inter-scales. To

make it fully invertible, split and invertible up-sampling operations are used. (3) Anomaly score map generation: an anomaly map is generated by associating a likelihood-based anomaly score to each pixel in the test image. (4) Anomaly segmentation: besides generating the anomaly map, we also propose to adapt the *a contrario* framework to obtain an automatic threshold by controlling the allowed number of false alarms

CNNs, often a VGG [36] or any variant of the ResNet [37] architectures, to extract a rich multi-scale image feature representation, by keeping the intermediate activation maps at different depths of the network. More recently, with the development of image vision transformers, some architectures such as ViT [38] and CaIT [21] are also being used, but in these cases, a single feature map is retrieved. The features generated by vision transformers are proven to better compress all multi-scale information into a single feature map volume, compared to the standard CNNs. However, this notion of a unique multi-scale feature map was challenged by MViT [39], which proved that combining the standard CNNs' idea of having multi-scale feature hierarchies with the transformer models, the transformers' features could be further enhanced. This work was followed by MViT2 [40], which was shown to outperform its predecessor and all other vision transformers in various tasks. In our work, we also propose to construct a multi-scale transformer architecture but following a different strategy. Specifically, we propose MS-CaIT, an architecture that employs CaIT transformers at different scales, independently pre-trained on ImageNet [14], and combines them as the encoder of a U-Net [41] architecture. Despite its simplicity, an ablation study presented in Sect. 5.2 shows that this combination strategy leads to better results than using each of the transformers alone, far better than other ResNet variants, and even better than MViT2. We conjecture that the fact that the CaIT networks were trained independently could be beneficial because it might give the network more flexibility to concentrate on different structures in each transformer.

As a final comment regarding the feature extraction stage, it is important to mention that, notwithstanding all the results

reported in Sect. 4 were obtained using exactly the same architecture configuration, the proposed architecture and the code are agnostic to the feature extractor and can be used with different extractors and different numbers of scales. For specific adaptation to certain situations and datasets, it could be better to use another extractor, for example, if computational power is very tight (e.g., in an industrial environment), probably at the expense of some decrease in performance.

3.2 Phase 2: Normalizing Flow

Normalizing flows [42] are a family of generative models that are trained by directly maximizing the log-likelihood of the input data and which have the particularity of learning a bijective mapping between the input distribution and the latent space. Using a series of invertible transformations, the NF can be run in both directions. The forward process embeds data into the latent space and can serve as a measure of the likelihood. The reverse process starts from a predefined distribution (usually a standard normal distribution) and generates samples following the learned data distribution. The rationale for using NFs in an anomaly detection setting is straightforward. The network is trained using only anomaly-free images, and in this way, it learns to transform the distribution of normal images into a white Gaussian noise process. At test time, when the network is fed with an anomalous image, it is expected that it will not generate a sample with a high likelihood, according to the white Gaussian noise model. Hence, a low likelihood indicates the presence of an anomaly.

This second phase is the only one that is trained. It takes the multi-scale representation generated by the feature extractor

as input and performs a sequence of invertible transformations on the data using the NF framework.

State-of-the-art methods following this approach are centered on designing or trying out different multi-scale feature extractors. In this work, we further improve the approach by proposing a new feature extractor and a multi-level deep feature integration method that aggregates the information from different scales using the well-known and widely used UNet-like [41] architecture. The U-shape comprises the feature extractor as the encoder and the NF as the decoder. We show in the ablation study of Sect. 5.1 that the U-shape is a beneficial aggregation strategy that further improves the results.

The 2D NF uses only invertible operations and treats each pixel location independently, only merging the information between different channels for each pixel location. As it is implemented as one unique computational graph, we obtain a fully invertible network. Moreover, optimizing the whole flow all at once has a crucial implication: the normalizing flow generates a mutually independent embedding not only within each scale but also across different scales, unlike other state-of-the-art flow-based methods. It is worth mentioning that in most works reported so far in the literature, the anomaly score is computed first at each scale independently, using a likelihood-based estimation, and finally merged by averaging or summing the result for each scale. Because of the lack of independence between scales, these final operations, although achieving very good performance, lack a formal probabilistic interpretation. The NF architecture proposed in this work overcomes this limitation; it produces statistically independent multi-scale features, for which the joint likelihood estimation becomes trivial.

Architecture. The U-shaped NF is compounded by a number of *flow stages*, each one corresponding to a different scale, whose size matches the extracted feature maps (see Fig. 2). For each scale starting from the bottom, i.e., the coarsest scale, the input is fed into its corresponding *flow stage*, which is essentially a sequential concatenation of *flow steps*. The output of this *flow stage* is then split in such a way that half of the channels are sent directly to the output of the whole graph, and the other half is up-sampled to be concatenated with the input of the next scale, and proceed in the same fashion. The up-sampling is also performed in an invertible way, as proposed in [43], where pixels in groups of four channels are reordered in such a way that the output volume has four times fewer channels and double width and height.

Each *flow step* has a size according to its scale and is compounded by Glow blocks [7]. Each step combines an affine coupling layer [5], a permutation using 1×1 convolutions, and a global affine transformation (ActNorm) [6]. The affine coupling layer uses a small network with the following layers: a convolution, a ReLU activation, and one more

convolution. Convolutions have the same amount of filters as the input channels, and kernels alternate between 1×1 and 3×3 .

To sum up, the U-shaped NF produces L white Gaussian embeddings $z^1, \dots, z^L$, one for each scale l , $z^l \in \mathbb{R}^{C_l \times H_l \times W_l}$. Here, we denote the spatial location by (i, j) , and by k the channel index in the latent tensors. Its elements

$$z_{ijk}^l \sim \mathcal{N}(0, 1), \quad (1)$$

$1 \leq i \leq H_l$, $1 \leq j \leq W_l$, $1 \leq k \leq C_l$, are mutually independent for any position, channel and scale i, j, k, l .

3.3 Phase 3: Likelihood-Based Anomaly Score

This phase of the method is to be used at test time when computing the anomaly map. It takes as input the embeddings $\{z_{ijk}^l\}$ and produces an anomaly map based on the likelihood computation. Thanks to the statistical independence of the features produced by the U-shaped NF, the joint likelihood of a test image under the anomaly-free image model is the product of the standard normal marginals. Therefore, to build this map, we associate to each pixel (i, j) in the test image, a likelihood-based anomaly score similar to those in [19, 35]:

$$AS(i, j) = 1 - \frac{1}{L} \sum_{l=1}^L \exp \left(-\frac{1}{2C_l} \sum_{k=1}^{C_l} (z_{ijk}^l)^2 \right), \quad (2)$$

where $\tilde{z}_{ijk}^l$ is a bilinearly upsampled version of z_{ijk}^l at the original $H \times W$ resolution.

Note that the exponential term of the above expression does not correspond exactly to the anomaly-free likelihood since, for each scale, an average in the channel direction is computed instead of the sum. The reason for this is mainly computational: using the sum produces extremely small values close to the floating point error. Using the sum has another undesirable effect: the impact of the different scales in the total anomaly score is not equalized, as the number of channels C_l may significantly differ. Although these modifications lead to an anomaly score that lacks a precise probabilistic interpretation, this computation produces high-quality anomaly maps and increases performance in practice.

In addition, in the following section, we introduce a more formal and statistically meaningful result, using the *a contrario* theory, that permits to derive an unsupervised detection threshold on the *number of false alarms (NFA)*, and produces an anomaly segmentation mask.

3.4 Phase 4: A Contrario Anomaly Segmentation

Besides generating the anomaly map discussed in the previous section, we present a new method for computing an

automatic segmentation of the anomalies based on the *a contrario* approach. In this approach, an estimate of the number of false alarms (NFA) is associated with each anomaly candidate, and detections are obtained by thresholding the NFA [8]. The proposed method has no parameters to tune and uses level sets organized into a tree structure to segment the anomalies automatically.

3.4.1 Number of False Alarms

The *a contrario* framework [8] is a multiple hypothesis testing methodology that has been successfully applied to derive unsupervised statistical detection thresholds in a wide variety of detection problems, such as alignments for line segment detection [44], clustering [45], image forgery detection [46], and even anomaly detection [47–49], to name a few. In [50], the authors revisit several anomaly detection methods from the perspective of the *a contrario* framework to derive unsupervised statistical detection thresholds. The approach we propose in this work follows the same rationale, as it exploits the statistical independence of the U-Flow embeddings to derive anomaly detection thresholds.

The *a contrario* framework is based on the non-accidentalness principle [51]. Given that we do not usually know what the anomalies look like, it focuses on modeling normality by defining a null hypothesis ($\mathcal{H}_0$), also called *background model*. Relevant structures are detected as large deviations from this model by evaluating how likely it is that an observed structure or event E would happen under $\mathcal{H}_0$. One of its most useful characteristics is that it provides an easily interpretable detection threshold based on controlling the NFA, defined as

$$\text{NFA}(E) = N_T \Pr_{\mathcal{H}_0}(E). \quad (3)$$

Here N_T denotes the number of events that are tested, and $\Pr_{\mathcal{H}_0}(E)$ is the probability of the event E being a realization of the background model. Besides providing a robust detection threshold, as it will be shown, the NFA value itself has a clear statistical meaning: it is an estimate of the expected number of times, among the tests that are performed, that such tested event could be generated by the background model [8]. In other words, an event E is ε -meaningful if its expected number of occurrences is less than ε under the normality assumption. Consequently, ε provides an upper bound on the expected number of false detections we can obtain on anomaly-free images. A low NFA value means that the observed pattern is too unlikely to be generated by the background model and, therefore, indicates a meaningful anomaly.

As the events E corresponding to anomalies can have arbitrary sizes and shapes, we must test all possible configurations of connected sets of pixels within every image. But this is computationally very inefficient and possibly even

impossible in a reasonable amount of time. Therefore, we propose to use the anomaly map to select the regions to be tested from the set of connected components of its level sets. These components are naturally ordered by inclusion, providing a convenient hierarchical representation that can be organized in a tree structure.

3.4.2 Detecting Anomalies from the Level Sets of the Anomaly Map

In image processing, level sets are the basis for a wide variety of morphological filters, and, as they are closely related to object boundaries, they have been used for edge detection [52, 53], registration [54, 55], image quantization [56], and segmentation [57], among others. All the information of a gray-valued image $u(i, j)$ is contained in a set of binary images obtained by thresholding at different levels. Since no information is lost, level sets provide a complete representation: any image can be easily reconstructed from the whole family of its (upper) level sets [58]

$$\mathcal{L}_\lambda = \{(i, j) \mid u(i, j) \geq \lambda\}. \quad (4)$$

We denote by $\mathcal{L}_{\lambda,c}$ the c -th connected component of $\mathcal{L}_\lambda$.

Level sets naturally define a hierarchical representation, ordered by their geometrical inclusion. In the case of the upper level sets, each connected component defined by a certain level is included in another connected component defined by a lower level. Therefore, they can be naturally embedded in a tree representation.

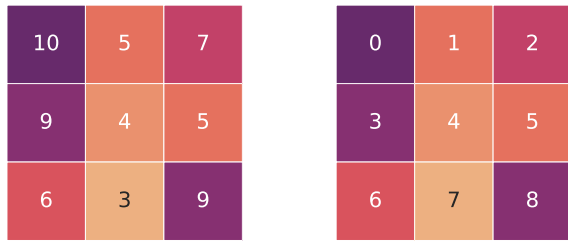
We treat all scales produced by the U-Flow independently and use these properties to build one *tree of connected components* for each scale. For simplicity, in this section, we present the method for one scale, and later in Sect. 3.4.3, we explain how to combine the results from different scales into a final set of detections. In the following, we omit the superscript l used to denote the embedding scale.

Specifically, for each scale, we propose to use the upper level sets of an image defined from the NF embeddings as

$$u(i, j) = \sum_{k=1}^C (z_{ijk})^2, \quad (5)$$

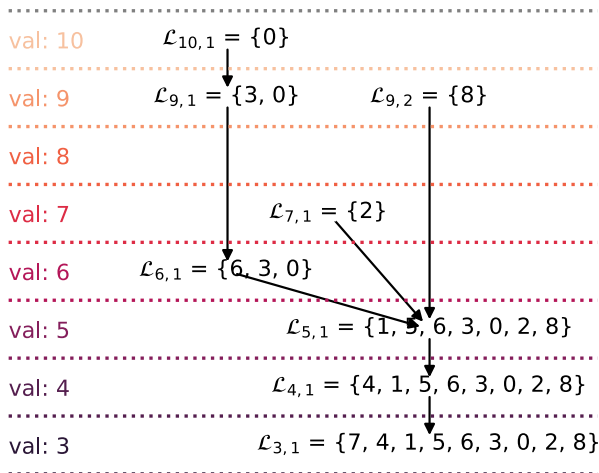
where C is the number of channels of the embedding. Note that for building the tree of connected components, as we are only interested in the level sets, using $u(i, j)$ is equivalent to using the Anomaly Score (2) for a single scale l since this score is an increasing function of $u(i, j)$.

Figure 3 shows a toy example that illustrates the procedure. Figure 3a represents the image u , with values $u(i, j)$ denoted by numbers inside each pixel. Figure 3b shows the raveled indexes that serve as labels for the tree nodes shown



(a) Example image u used to build the tree. Each pixel has its value annotated.

(b) Same image u , annotated with the raveled index of each pixel.



(c) Example tree of connected components built from the upper level sets of (a). Each node is represented by a set of indexes from (b), and each row corresponds to a different level set, according to the values of (a).

Fig. 3 Tree of connected components of the upper level sets: the hierarchical representation based on the level sets used to retrieve the most significant regions to be tested for anomaly segmentation

in Fig. 3c. The tree is built so that the nodes are given by the connected components of the level set $\mathcal{L}_{\lambda,c}$, and the edges represent the inclusion, connecting level set components from two different levels.

The tree of connected components allows us to significantly reduce the number of image regions we effectively test. Instead of computing the NFA of all possible N_T connected regions of any shape and size, which would be intractable, we will limit this computation to the connected components of the upper level sets in the tree. By construction, these regions are expected to be good candidates for anomalies.

To compute the NFA of a connected component $\mathcal{L}_{\lambda,c}$ in the tree, we need to evaluate the probability of false alarm (PFA), i.e., the probability of occurrence of a region like the observed $\mathcal{L}_{\lambda,c}$ under the background model of normality. In the absence of anomaly, the embedding produced by the U-Flow model is $z_{ijk}^l \sim \mathcal{N}(0, 1)$, i.i.d. Therefore, variables

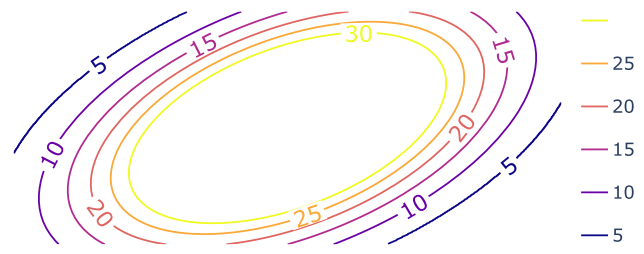


Fig. 4 Example level lines of a branch of the tree of connected components

$u(i, j)$ as defined in (5) are identically distributed as a *Chi-Squared* distribution of order C . Then, as the minimum pixel value in the connected component $\mathcal{L}_{\lambda,c}$ is λ , it follows that its probability of false alarms is given by

$$\begin{aligned} \text{PFA}(\mathcal{L}_{\lambda,c}) &= \Pr\left(\min_{(i,j) \in \mathcal{L}_{\lambda,c}} u(i, j) \geq \lambda\right) \\ &= \left(1 - \frac{\text{CDF}(\lambda)}{\chi^2(C)}\right)^{|\mathcal{L}_{\lambda,c}|}, \end{aligned} \quad (6)$$

where $|\mathcal{L}_{\lambda,c}|$ denotes the number of pixels in $\mathcal{L}_{\lambda,c}$. This measure of meaningfulness of a region presents a natural balance between growing the regions (and with that increasing $|\mathcal{L}_{\lambda,c}|$), and reducing the minimum value of its pixels (i.e., reducing λ). This balance is desirable to delineate the abnormal regions well.

For efficiency purposes, instead of computing the *CDF* in (6), we use the Chernoff bound [59] for a $\chi^2(C)$ distribution obtaining the final base 10 logarithm of the PFA value:

$$\begin{aligned} \log(\text{PFA}(\mathcal{L}_{\lambda,c})) &= \\ &= |\mathcal{L}_{\lambda,c}| \frac{C}{2 \ln(10)} \left(1 + \ln\left(\frac{\lambda}{C}\right) - \frac{\lambda}{C}\right). \end{aligned} \quad (7)$$

Once we have computed the $\log(\text{PFA})$ values of all connected components in the tree, to obtain the final regions, we iteratively perform the following *prune* and *merge* procedures until convergence (until the tree does not change anymore).

Procedure 1: PFA Prune. This procedure aims to filter a set of nested connected components, keeping only the most significant one. To do so, we directly use the $\log(\text{PFA})$ value of each node (each connected component), and select the lowest one. Starting from each leaf, we identify a connected section of the tree where all nodes have exactly one predecessor (except for the starting leaf itself). The concentric level lines defined by such a branch are illustrated by the example in Fig. 4. In other words, we identify which of these connected components is the most significant one, as it may better delineate the anomalous region. After determining which node to preserve, the tree is pruned so that just the chosen node is kept, and all other branch nodes are removed.

In the example of Fig. 3c, we would consider the branch $b_1 = \{\mathcal{L}_{10,1}, \mathcal{L}_{9,1}, \mathcal{L}_{6,1}\}$, evaluate the $\log(\text{PFA})$ value for each node, and prune the tree, keeping only the one with lowest value, i.e., the most significant region, denoted as $\mathcal{L}_{b_1}$.

Procedure 2: PFA Merge. The second procedure consists of merging leaf nodes with the same successor in case the latter is more significant than all others. In this case, all leaf nodes are removed from the tree, and we only keep their successor. In the example of Fig. 3c, this procedure would analyze if the node $\mathcal{L}_{5,1}$ (the successor) is more significant than $\mathcal{L}_{b_1}$ (the leaf predecessor node resulting from *Procedure 1*), $\mathcal{L}_{9,2}$, and $\mathcal{L}_{7,1}$. In case $\mathcal{L}_{5,1}$ has the lowest $\log(\text{PFA})$ value between all these nodes, the other nodes are removed from the tree. Otherwise, this procedure makes no changes to the tree.

Note that in the case some actual merge is performed in *Procedure 2*, it could generate a new branch with more than one node, starting from a new leaf, where all nodes in the branch have only one predecessor (except the starting leaf itself). Therefore, we iteratively apply both procedures until no more changes are performed over the tree. At the end of this iterative process, we use the resulting leaf nodes of the tree as the final regions and assign them the $\log(\text{PFA})$ value of the corresponding connected component, generating a $\log(\text{PFA})$ heatmap per scale.

3.4.3 $\log(\text{NFA})$ Computation

The scales are merged by upsampling the resulting $\log(\text{PFA})$ heatmaps at different scales l , up to the original input size (H, W) , and keeping the minimum value at each pixel position (i, j) :

$$\log(\text{PFA}(i, j)) = \min_l \left\{ \uparrow^{H,W} \left(\log(\text{PFA}^l) \right) (i, j) \right\}, \quad (8)$$

where $\uparrow^{H,W}(\cdot)$ represents the bilinear upsample up to the original size.

Finally, for the number of tests N_T , we need to quantify all possible regions, or arrangements of connected pixels of any size and shape, located in any part of the image. Considering 4-connectivity, these groups of connected pixels correspond to the figures called *polyominoes*, and a good approximation for the number of polyominoes of r pixels is given by $\alpha\beta^r/r$, with $\alpha = 0.316915$, and $\beta = 4.062570$ (see [60, 61]). Therefore, considering all possible region sizes and shapes in all possible spatial positions of all scales results in

$$N_T = \sum_l H_l W_l \sum_{r=1}^{H_l W_l} \alpha \frac{\beta^r}{r}. \quad (9)$$

Finally, the $\log(\text{NFA})$ value is given by

$$\log(\text{NFA}(i, j)) = \log(N_T) + \log(\text{PFA}(i, j)). \quad (10)$$

4 Experimental Results

The proposed approach was tested and compared with state-of-the-art methods, by conducting extensive experimentation on several datasets: **MVTec-AD** [23], **BeanTech** (BT) [24], **LGG-MRI** (MRI) [25], and **ShanghaiTech Campus** (STC) [26].

MVTec-AD is the most widely used benchmark for anomaly detection, as most state-of-the-art anomaly detection methods report their performances on this dataset. It consists of 15 categories of textures and objects, simulating real-world industrial quality inspection scenarios with 5,000 images. Each category has normal (i.e., anomaly-free) images for training, and the testing set consists of images with different kinds of anomalies and some more normal images. **BeanTech** is also a real-world industrial anomaly dataset. It contains 2830 images of three categories, including normal and defective images.

Besides testing the proposed method on the industrial inspection task (which is the motivation and focus of this work), we also include experimentation with data from other fields to demonstrate the generalization capability of our method. To do so, we test and compare our method on datasets from the medical and surveillance fields. **LGG-MRI** contains images of lower-grade gliomas (brain tumors), with images of the tumors' evolution over time, from 110 patients of 5 institutions, taken from The Cancer Genome Atlas. Lastly, the **ShanghaiTech Campus** dataset contains frames extracted from videos of 13 different scenes of surveillance cameras of the mentioned campus.

For assessing the anomaly maps defined in (2), we adopt the *AUROC* metric (the area under the receiver operating characteristic curve) at a pixel level, as it is the most widely used metric in the literature. We also consider *AUPRO* (the area under the per-region-overlap curve) since it ensures that both large and small anomalies are equally important. For assessing the anomaly masks (after thresholding), we use the *mIoU* metric (mean intersection over union). Finally, even though the image-level anomaly detection task is not the primary focus of this work, we have included the image-level *AUROC* results in Appendix A because we understand that it is important to check that any anomaly localization method should also perform well when used to classify an entire image as anomalous or normal (the detection task).

4.1 Results on the MVTec-AD Dataset

Before proceeding to present the results on this dataset, it is worth mentioning that since the trained models for FastFlow, CFlow, and CS-Flow are not available, it was necessary to re-train them for all the categories. To ensure a fair comparison, we performed numerous training runs until the obtained *AUROC* were comparable to the ones reported in the original papers. Note that for FastFlow and CFlow, a different set of hyper-parameters is chosen for each category, sometimes varying even the number of *flow steps* and the image resolution, while for our method, the architecture is always kept the same.

The results obtained for the anomaly maps assessment in terms of *AUROC* and *AUPRO* are shown in Tables 1 and 2, respectively. U-Flow achieves state-of-the-art results, outperforming all previous methods on average for *AUROC*. Considering the *AUPRO* metric, our results rank first among the flow-based methods and second overall, achieving the best results for several categories. It is worth emphasizing that FastFlow and CFlow results are reported using different hyperparameters that even change the architecture, and this is somehow unfair in their favor. Our results are always superior to CS-Flow, which also uses a multi-scale strategy inside the NF.

In addition, besides obtaining excellent results for *AUROC* and *AUPRO* in the anomaly localization task, our method presents another significant advantage with respect to all others: it produces a fully unsupervised segmentation of the anomalies and significantly outperforms its competitors, as shown in the next section.

For visual assessment, we include Fig. 5, which presents typical example results of anomalous images from various categories and their comparison with other methods, and Fig. 6, which shows typical examples of normal images in order to check that the anomaly maps present low values and that the detection method does not produce false positives.

4.1.1 Segmentation results

Providing an operation point is crucial in almost any industrial application. Most recent deep learning industrial anomaly detection methods in the literature focus on generating anomaly maps and evaluating them using the *AUROC* metric; they do not provide detection thresholds or anomaly segmentation masks. Some methods do provide segmentation results, but these are not consistent with a realistic scenario: they are obtained using an *oracle*-like threshold that is computed by maximizing some metric over the test images [28, 31, 35].

In this section, we report results concerning anomaly segmentation based on the standard unsupervised threshold for meaningful events: $NFA \leq 1$ ($\log(NFA) \leq 0$). As detailed in

Sect. 3.4.1, this threshold implies that, theoretically, we allow one false anomaly detection per image on average. Moreover, this threshold can be tuned for any application-specific needs. For instance, in an industrial environment with the goal of detecting sample defects, an average value of one false detection per image may not be a good choice, as it would raise too many false alarms in normal images (one per image, on average). Instead, in this case, the threshold could be customized to anticipate, for example, a designated frequency of false alarms within a defined timeframe. However, as we will show next, the proposed method is inherently very robust to the threshold choice, and a much more restrictive threshold on the false positives can be considered without affecting the true positives' rate.

For evaluating the anomaly segmentation performance, we propose to use the *mIoU* metric. Compared to *IoU*, using the average of all images (*mIoU*) prevents large anomaly regions from having more weight in the overall metric than small ones, and therefore, this metric better reflects how good the methods are in detecting all anomalies. The results for the *mIoU* metric are shown in Table 3. We limit this comparison to the flow-based methods. As the state-of-the-art methods to which we compare do not provide detection thresholds, we adopt two strategies: (i) we compute an *oracle*-like threshold which maximizes the *mIoU* for the testing set, and (ii) we use a *fair* strategy that only uses training data to find the threshold. In the latter, the threshold is set to allow on average one false positive in the training (anomaly-free) images, as it would be analogous to setting $NFA \leq 1$ false alarm.

Visual examples of the obtained detections can also be found in Fig. 5 for anomalous images, and in Fig. 6 for normal images (which present no detections).

As seen in Table 3, our automatic thresholding strategy significantly outperforms all others, even when compared with their *oracle*-like threshold. Additionally, it is important to note that our results obtained with the oracle-like and automatic thresholds are very close. This demonstrates the validity of the proposed statistical derivation and the inter-scale independence achieved by the proposed architecture. To further illustrate this point and to show the robustness of the anomaly detection method with respect to the threshold on the NFA, Fig. 7 depicts the *mIoU* for all the MVTec-AD categories, as a function of $-\log(NFA)$ threshold. These graphs show that the unsupervised threshold $\log(NFA) \leq 0$ is always near the optimal point (the oracle threshold), corresponding to the maximum *mIoU*, that is reported in Table 3 (noted as “Oracle threshold $\rightarrow$ Ours”). Not only are the oracle and the automatic thresholds close, but the variation in *mIoU* is very low in a wide range of possible thresholds, showing that this detection strategy is robust and parameter-free in practice.

As a verification, we additionally include a numerical evaluation of the proposed method's performance under different

Table 1 MVTec-AD results: pixel-level *AUROC*

Category	Patch SVDD	SPADE	PaDiM	Cut Paste	Patch Core	PEFM	Fast Flow	CFlow	CS- Flow	U-Flow (ours)
Carpet	92.60	97.50	99.10	98.30	98.90	99.00	99.40	99.25	95.23	99.42
Grid	96.20	93.70	97.30	97.50	98.70	98.48	98.30	98.99	94.46	98.49
Leather	97.40	97.60	98.90	99.50	99.30	99.24	99.50	99.66	89.32	99.59
Tile	91.40	87.40	94.10	90.50	95.60	95.19	96.30	98.01	90.71	97.54
Wood	90.80	88.50	94.90	95.50	95.00	95.27	97.00	96.65	86.29	97.49
Av. texture	93.68	92.94	96.86	96.26	97.50	97.44	98.10	98.51	91.20	98.51
Bottle	98.10	98.40	98.30	97.60	98.60	98.11	97.70	98.98	88.52	98.65
Cable	96.80	97.20	96.70	90.00	98.40	96.58	98.40	97.64	96.08	98.61
Capsule	95.80	99.00	98.50	97.40	98.80	97.94	99.10	98.98	98.13	99.02
Hazelnut	97.50	99.10	98.20	97.30	98.70	98.78	99.10	98.89	96.13	99.30
Metal nut	98.00	98.10	97.20	93.10	98.40	96.89	98.50	98.56	95.98	98.82
Pill	95.10	96.50	95.70	95.70	97.10	96.67	99.20	98.95	95.83	99.35
Screw	95.70	98.90	98.50	96.70	99.40	98.93	99.40	98.86	98.28	99.49
Toothbrush	98.10	97.90	98.80	98.10	98.70	98.28	98.90	98.93	97.39	98.79
Transistor	97.00	94.10	97.50	93.00	96.30	96.58	97.30	97.99	96.34	97.87
Zipper	95.10	96.50	98.50	99.30	98.80	98.29	98.70	99.08	95.82	98.60
Av. objects	96.72	97.57	97.79	95.82	98.32	97.71	98.63	98.69	95.85	98.85
Av. total	95.71	96.03	97.48	95.97	98.05	97.62	98.45	98.63	94.30	98.74

The two best results for each row are in bold. Comparison with the best performing methods: Patch SVDD [27], SPADE [29], PaDiM [30], Cut Paste [33], Patch Core [31], PEFM [62], Fast Flow [19], CFlow [35], and CS-Flow [20]. Our method outperforms all previous methods, with an average value of 98.74%

Table 2 MVTec-AD results: pixel-level *AUPRO*

Category	SPADE	PaDiM	PEFM	FastFlow	CFlow	CS-Flow	U-Flow (ours)
Carpet	94.70	96.20	96.75	97.30	97.70	83.55	98.49
Grid	86.70	94.60	97.21	94.79	96.08	78.83	95.46
Leather	97.20	97.80	98.91	99.16	99.35	81.05	98.40
Tile	75.90	86.00	91.10	86.90	94.34	72.97	93.01
Wood	87.40	91.10	95.77	94.14	95.79	60.49	95.80
Av. texture	88.38	93.14	95.95	94.46	96.65	75.38	96.23
Bottle	95.50	94.80	95.92	91.55	96.80	64.14	95.64
Cable	90.90	88.80	97.73	94.38	93.53	84.07	93.73
Capsule	93.70	93.50	92.11	91.22	93.40	89.30	95.02
Hazelnut	95.40	92.60	97.99	97.31	96.68	82.77	97.35
Metal nut	94.40	85.60	93.88	91.05	91.65	76.93	94.32
Pill	94.60	92.70	96.18	96.80	95.39	75.61	97.18
Screw	96.00	94.40	95.73	67.00	95.30	92.21	97.33
Toothbrush	93.50	93.10	96.21	92.22	95.06	78.48	88.80
Transistor	87.40	84.50	90.84	91.76	81.40	91.87	91.88
Zipper	92.60	95.90	96.45	95.19	96.60	85.84	95.47
Av. objects	93.40	91.59	95.30	91.05	93.58	82.12	94.67
Av. total	91.73	92.11	95.52	92.18	94.60	79.87	95.19

The two best results for each category are shown in bold. Comparison with the best-performing method available: SPADE [29], PaDiM [30], PEFM [62], and the best flow-based methods: FastFlow [19], CFlow [35], and CS-Flow [20]. Patch Core [31] is excluded from the table as it only reports a total average of 93.50%. Our method also achieves state-of-the-art performance for this metric, even obtaining the best result for some categories

Table 3 Segmentation *mIoU* comparison for MVTec-AD, with the best flow-based methods in the literature: FastFlow [19], CFlow [35], and CS-Flow [20], for the oracle-like and fair thresholds defined in Sect. 4.1.1

Category	Oracle threshold				Fair threshold			
	Fast Flow	C- Flow	CS- Flow	Ours	Fast Flow	C- Flow	CS- Flow	Ours
Carpet	0.5451	0.4519	0.3521	0.5664	0.4267	0.2393	0.3420	0.5663
Grid	0.4664	0.4077	0.3498	0.4316	0.2235	0.3413	0.3488	0.4307
Leather	0.5483	0.4818	0.2986	0.4242	0.5241	0.3096	0.2823	0.4183
Tile	0.5564	0.3705	0.4906	0.6229	0.4342	0.3105	0.2544	0.6188
Wood	0.4792	0.3767	0.3400	0.5523	0.4584	0.2433	0.0806	0.5427
Av. Texture	0.5191	0.4177	0.3662	0.5195	0.4134	0.2888	0.2616	0.5154
Bottle	0.5750	0.6503	0.4098	0.6617	0.4053	0.6434	0.3935	0.6564
Cable	0.5375	0.4009	0.5018	0.6514	0.4807	0.2299	0.3668	0.6491
Capsule	0.3439	0.3040	0.2739	0.4182	0.3160	0.3002	0.2669	0.4124
Hazelnut	0.4723	0.5610	0.4654	0.6546	0.4671	0.4831	0.2747	0.6511
MetalNut	0.5801	0.3981	0.4631	0.6307	0.5801	0.2998	0.4165	0.6267
Pill	0.4616	0.3121	0.2324	0.4815	0.4051	0.3096	0.2260	0.4803
Screw	0.2562	0.3039	0.3137	0.3811	0.2375	0.2755	0.3049	0.3726
Toothbrush	0.3625	0.3347	0.3160	0.4188	0.3007	0.2416	0.3016	0.4121
Transistor	0.6337	0.6000	0.6622	0.7381	0.0597	0.2981	0.6516	0.7354
Zipper	0.5772	0.5001	0.4092	0.5460	0.4184	0.2403	0.4082	0.5395
Av. Object	0.4800	0.4365	0.4048	0.5582	0.3671	0.3322	0.3611	0.5536
Total	0.4930	0.4302	0.3919	0.5453	0.3825	0.3177	0.3279	0.5408

The two best results for each row are in bold. Our method largely outperforms all others, and even exhibits a better performance comparing the proposed automatic threshold with their oracle-like threshold

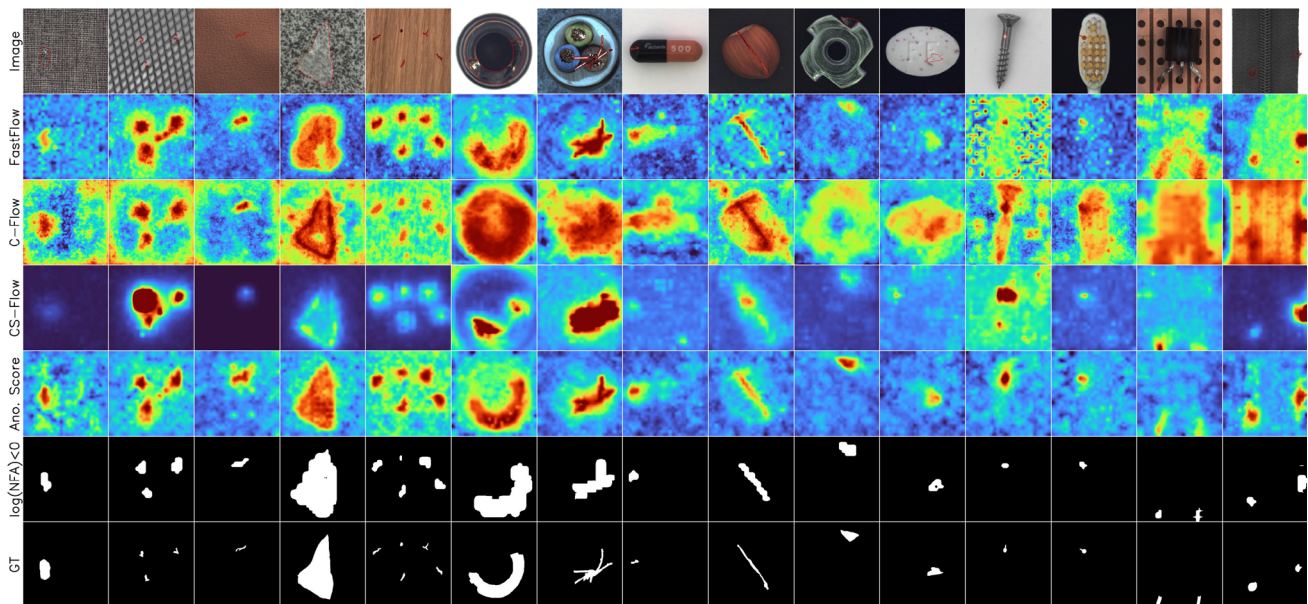


Fig. 5 Example results for all MVTec categories. The first row shows the example images with the ground truth over-imposed in red. The results for FastFlow, CFlow, and CS-Flow are shown in the second, third, and fourth rows. The next two rows correspond to our method: the anomaly score defined in (2), and the segmentation obtained with the automatic threshold $\log(\text{NFA}) < 0$. The last row presents the segmentation masks for an easy comparison. While other methods achieve

a very good performance, in some cases, they present artifacts and over-estimated anomaly scores. Our anomaly score achieves very good visual and numerical results, spotting anomalies with high confidence. Finally, the segmentation with the automatic threshold on the NFA is also able to spot and accurately segment the anomalies. All detections of these examples exhibit very low $\log(\text{NFA})$ values, ranging from -50 to -1515

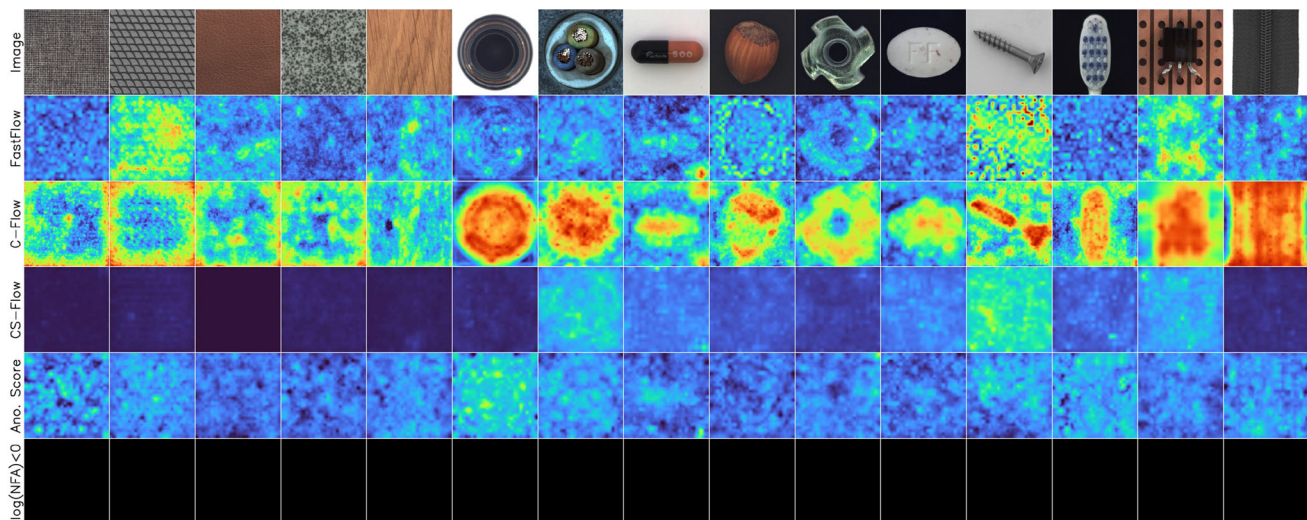


Fig. 6 Normal image examples for all MVTec-AD categories. As can be seen, we always predict low values in the anomaly maps, and no detections are made

false alarm thresholds in the Supplementary Material. When allowing one false alarm per image on average (with the threshold set as $\log(NFA) \leq 0$), the resulting average *mIoU* was found to be 0.541. Similarly, when permitting one false alarm per one million images on average (threshold $\log(NFA) \leq -6$), the obtained *mIoU* was almost identical, at 0.540. The robustness of our method to threshold variations can be attributed to its underlying approach. It utilizes a hierarchical structure based on the set of connected components derived from level sets, identifying maximal significant regions, defined as regions that potentially encompass lesser significant ones and are not encompassed by any higher significant region. Consequently, our method does not explicitly evaluate all potential regions. This characteristic contributes to computational efficiency and enhances the robustness of threshold selection. Each detected region encompasses a set of regions, and the ultimately identified regions are large deviations from the background model. Finally, we conducted detections on all anomaly-free test images to evaluate the empirical false alarm rate. Out of 467 images analyzed, 6 exhibited false alarms, of which 4 were identified as genuine yet unlabeled anomalies. The remaining 2 instances were confirmed as true false alarms, attributable to significant deviations from the null hypothesis observed in the normalizing flow embedding. It is worth mentioning that these two false alarms are two sets of detections, as discussed before, suggesting that the actual average false alarm rate could be closer to the theoretical one. We refer the reader to the Supplementary Material for a more detailed analysis.

4.2 Results on Other Datasets

In this section, we present the results obtained on the other datasets: BeanTech (BT), LGG MRI (MRI), and ShanghaiTech Campus (STC). Although our work is motivated by the industrial inspection task, we also evaluate the performance on datasets that are very different from that scenario. The results reported here demonstrate the robustness and generalization capability of the proposed approach (c.f. Table 5). For all datasets, we obtained excellent results, reaching top performance for almost all metrics and datasets. Again, all results were obtained by training the other methods with different hyperparameters to achieve the best possible results. In cases where some combinations of metrics and datasets were reported in the original articles, we confirmed that the same results had been obtained. Typical qualitative results are shown in Fig. 8.

4.3 Implementation Details and complexity

The method was implemented in PyTorch [63], using PyTorch Lightning [64]. The NFs were implemented using the FrEIA Framework [65], and for all tested feature extractors, we used PyTorch Image Models [66]. In all cases, training was performed in a GeForce RTX 2080 Ti.

For the MS-CaIT, the input sizes are 448x448 and 224x224 pixels. The normalizing flow has 2 *flow stages* with 4 *flow steps* each.

As mentioned, our method only uses four *flow steps* in each scale. As a result, it has fewer trainable parameters than FastFlow and CFlow for the same feature extractors, as shown in Table 4.

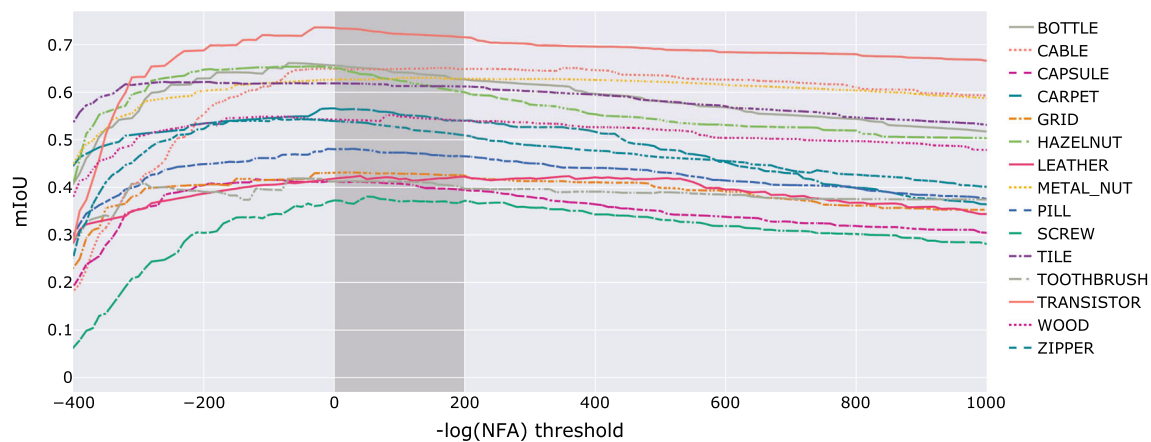


Fig. 7 Segmentation results (*mIoU*) as a function of the $-\log(\text{NFA})$ threshold for all MVtec-AD categories. The automatic threshold $-\log(\text{NFA}) = 0$ is always very close to the optimal threshold for each category (which corresponds to the maximum of each curve). In addition,

there is a wide range of thresholds, approximately indicated with the grayed-out region in the chart, for which the performance remains almost the same, indicating that this strategy is robust and, in practice parameter-free

Table 4 Complexity analysis: comparison of the number of trainable parameters for different feature extractors and methods

Feature Extractor	Fast Flow	CFlow	CS- Flow	U-Flow (ours)
ResNet18	4.9 M	5.5 M	—	4.3 M
WideResnet50	41.3 M	81.6 M	—	34.8 M
EfficientNet	—	—	275 M	—
CaIT M48	14.8 M	10.5 M	—	8.9 M
MS-CaiT	—	—	—	12.2 M

The method with the least number of trainable parameters for each feature extractor is in bold

Although all the results reported in this section were obtained using MS-CaiT as a feature extractor, we stress that the method and the code are developed agnostic to the feature extractor and can be easily changed according to the user's preference. In the following section, we present some results testing different feature extractors. In those cases, the number of scales and the volumes' sizes could vary depending on the chosen feature extractor.

5 Ablation Study

In this section, we study and provide unbiased assessments regarding some of the contributions of this work: the significance of the U-shaped architecture and the benefits of the multi-scale transformer feature extractor MS-CaiT. Both results are shown all together in Table 6 and explained in the following sections.

5.1 Ablation: U-Shape

One of the contributions of the presented work is proposing a multi-level integration mechanism by introducing the well-known U-shape architecture design to the normalizing flow's

framework. To demonstrate that this architecture better integrates the information of the different scales, we compare the results obtained in terms of *AUROC* against a modification of the architecture, in which each *flow stage* runs in parallel, and the per-scale anomaly maps are merged at the end by just averaging them, as done by other methods. Additionally, we compare the results obtained using each scale separately. The results, presented in the top half of Table 6, show that the U-merging strategy improves the performance in almost all cases. Furthermore, this strategy, where the output of one scale inputs the next one, allows the use of less *flow steps* for each scale, resulting in a network with fewer parameters, as shown in Sect. 4.3.

5.2 Ablation: feature extraction with MS-CaiT

From the upper half of Table 6, it is also clear that utilizing image transformers at various scales in MS-CaiT significantly improves the results, even if each of them already provides a multi-scale representation. Note that both scale-merging strategies outperform the results obtained by each individual transformer.

In addition, in this section, we compare the results obtained by the proposed MS-CaiT feature extractor with

Table 5 Results for LGG MRI [25], ShanghaiTech Campus (STC) [26], and BeanTech (BT) [24] datasets

	LGG MRI	STC	BT01	BT02	BT03	BT AV.
AUROC						
FastFlow	82.51	95.51	95.98	96.69	99.32	97.33
CFlow	86.41	96.90	94.29	96.13	99.58	96.67
CS-Flow	81.82	86.68	91.66	94.89	99.10	95.22
U-Flow (ours)	93.55	87.19	97.70	96.93	99.76	98.13
AUPRO						
FastFlow	47.48	73.67	76.23	62.39	96.60	78.41
CFlow	59.47	82.05	60.19	54.72	98.32	71.08
CS-Flow	37.30	54.04	73.75	47.18	94.45	71.79
U-Flow (ours)	71.74	47.54	80.06	66.49	98.83	81.79
mIoU Oracle						
FastFlow	0.2181	0.4840	0.4813	0.2929	0.9297	0.5680
CFlow	0.2182	0.4831	0.4267	0.2782	0.9323	0.5457
CS-Flow	0.2181	0.4895	0.3776	0.2439	0.9390	0.5202
U-Flow (ours)	0.2279	0.5052	0.4880	0.2620	0.9414	0.5638
mIoU Fair						
FastFlow	0.1947	0.4703	0.4408	0.2890	0.9129	0.5476
CFlow	0.2175	0.4831	0.4227	0.2686	0.9149	0.5354
CS-Flow	0.2060	0.4211	0.3522	0.1353	0.8738	0.4538
U-Flow (ours)	0.1467	0.4950	0.4667	0.2448	0.9330	0.5482

The best method for each dataset is in bold. Comparison with best-performing flow-based models: FastFlow [19], CFlow [35], and CS-Flow [20]. Our method obtains the best results for almost all metrics and datasets, outperforming the other methods. Both *AUROC* and *AUPRO* refer to the pixel-level metric (localization task). For *mIoU*, we present the results using both the “Fair” and the “Oracle”-like thresholds

Table 6 Ablation results

	Carp	Grid	Leat	Tile	Wood	Bott	Cable	Caps	HNut	MNut	Pill	Screw	Toot	Tran	Zipp	Total
Scale 1	99.08	97.40	99.32	94.97	93.45	98.33	98.11	98.87	99.09	97.89	98.67	99.44	98.74	96.46	98.55	97.89
Scale 2	99.12	97.09	99.42	96.47	96.26	97.23	97.60	98.07	98.63	97.86	98.62	99.18	97.86	97.21	97.29	97.86
Avg	99.44	98.25	99.52	97.27	96.40	98.61	98.50	98.85	99.16	98.29	99.12	99.50	98.78	97.66	98.69	98.54
Ours	99.42	98.49	99.59	97.54	97.49	98.65	98.61	99.02	99.30	98.82	99.35	99.49	98.79	97.87	98.60	98.74
ResNet	98.80	98.26	99.37	94.53	94.50	98.00	96.96	98.46	98.63	96.70	97.45	98.01	98.20	98.38	97.43	97.58
Arch	Wide	r18	r18	Wide	r18	r18	Wide	Wide	Wide	Wide	Wide	Wide	r18	Wide	Wide	–
F. steps	6	6	4	8	6	4	4	4	4	4	4	6	6	4	4	–
MViT2	98.74	97.73	99.20	93.83	94.75	92.79	96.98	98.83	97.50	96.39	97.36	97.69	87.14	97.00	96.39	96.15

The best variant for each category is in bold. The **middle row** (in-between horizontal lines), which shows the pixel-level *AUROC* results of our proposed method (U-Flow), serves for comparison for the top and bottom halves of the table. **Top half**: ablation study for the scale-merging strategy. Results using the anomaly scores generated by each scale independently, and a naive way of merging them (average). **Bottom half**: ablation study for the feature extractor. The proposed method (that uses MS-CaIT) is compared with ResNet and MViT2 feature extractors. For each category, we show the *AUROC* of the best variant we could obtain, varying several hyper-parameters, such as the architecture (wide-resnet-50 or resnet-18) and the number of *flow steps*

the most common ResNet variants used in the literature: *ResNet-18* and *Wide-ResNet-50*, and the multi-scale transformer MViT2. For all feature extractors, we extract features from all scales and proceed in the exact same way as presented before. Table 6’s bottom half makes it clear that the MS-CaIT feature extractor performs far better than ResNet variants and MViT2. Note that, in favor of the ResNet extrac-

tors, for each category, we picked the best result obtained by both variants and also varied several hyper-parameters, for example, the amount of *flow steps* in each scale, while for MS-CaIT we always use the same exact architecture. Finally, note that our feature extractor always achieves better performance compared to MViT2, which is also a transformer with a multi-scale hierarchy, probably getting some benefit from

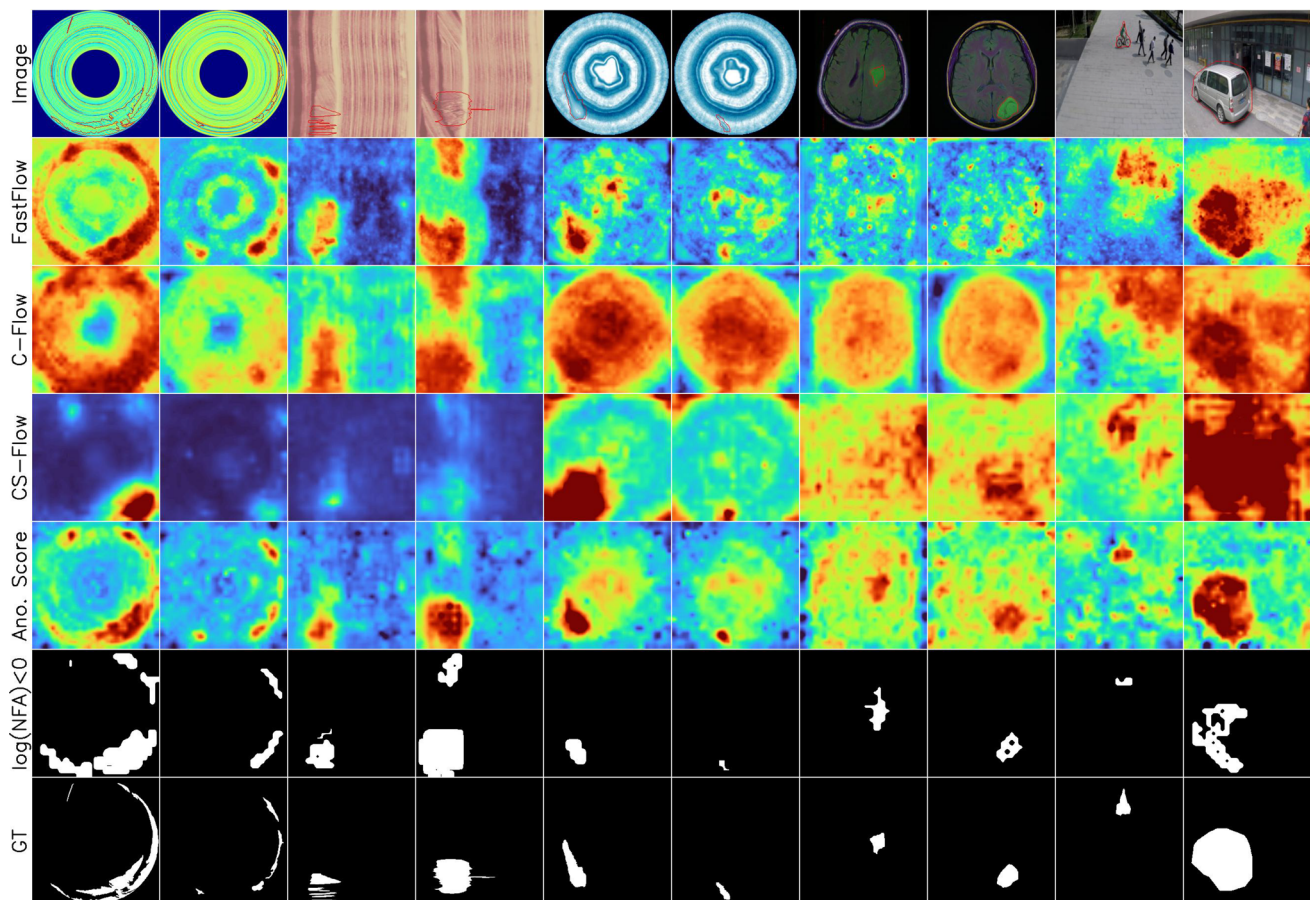


Fig. 8 Examples of typical results on BeanTech, LGG MRI, and STC datasets. The first row shows the original images with over-imposed ground truth. The second, third, and fourth rows show the results of FastFlow, CFlow, and CS-Flow for comparison. The following two rows are the outputs of our method: the anomaly score defined in (2), and

the anomaly segmentation with $\log(\text{NFA}) < 0$, and the last row is the ground truth. Again, our method achieves very good visual and numerical results and is able to detect the anomalies with great confidence. All detections of these examples exhibit very low $\log(\text{NFA})$ values, ranging from -56 to -1586

the training independence for each scale. The hyperparameter search for ResNet and MViT2 variants was performed with Optuna [67], using the TPE [68] sampler.

6 Conclusion

In this work, we introduced a novel anomaly detection method that achieves state-of-the-art results on various datasets and even outperforms them in most cases. By using modern techniques with outstanding performance, such as transformers and normalizing flows, we developed a method that exploits their characteristics to integrate them with classic statistical modeling. Our approach consists of four phases that follow one another, and we presented a clear and compelling contribution for each one: (i) We propose a new feature extractor using pre-trained transformers to build a multi-scale representation. (ii) We integrate the U-shape

architecture into the normalizing flow framework, creating a complete invertible architecture that settles the theoretical foundations for the NFA computation by ensuring independence in the embedding not only intra-scale but also inter-scales. (iii) We compute a likelihood-based anomaly score for each pixel with state-of-the-art performance in various datasets. And (iv) we propose a new anomaly detection method that permits to derive an unsupervised threshold based on the *a contrario* framework; this method exploits the statistical independence of the U-Flow embeddings to build a background model that produces excellent anomaly segmentation results.

The proposed approach was extensively evaluated and compared with the top-performing methods in the literature using different metrics, obtaining state-of-the-art results and, in most cases, even outperforming all previous methods. Finally, the method was also applied to several datasets

from different domains with excellent results, demonstrating a strong generalization capability.

Appendix A Image-Level Results

As mentioned in Sect. 4, this work focuses on the anomaly localization task. Nevertheless, we understand it is important to check that the proposed method also behaves well on the image-level detection task. Therefore, we present in Table 7 the results for the image-level *AUROC* (detection task). To

compute an image-level score, we simply used the maximum value of the anomaly score at the pixel level, defined in Sect. 3.3 (Eq. (2)). This simple strategy already achieves very good results: compared with all flow-based methods, our method ranks second, only behind FastFlow. However, it is worth noting that FastFlow actually uses different hyperparameters for each category, even changing the architecture, which is an unfair test-set tuning.

For completeness, in Table 8 we also include the image-level results for all other tested datasets. Our method ranks first on average for LGG MRI and BeanTech.

Table 7 MVTec-AD results: image-level *AUROC*.

Category	P.SVDD	SPADE	Cut- Paste	Patch Core	PEFM	Fast Flow	CFlow	CS-Flow	U-Flow (ours)
Carpet	92.90	98.60	100.0	98.70	100.0	100.0	100.0	100.0	100.0
Grid	94.60	99.00	96.20	98.20	96.57	99.70	97.60	99.00	99.75
Leather	90.90	99.50	95.40	100.0	100.0	100.0	97.70	100.0	100.0
Tile	97.80	89.80	100.0	98.70	99.49	100.0	98.70	100.0	100.0
Wood	96.50	95.80	99.10	99.20	99.19	100.0	99.60	100.0	99.91
Av. texture	94.54	96.54	98.14	98.96	99.05	99.94	98.72	99.80	99.93
Bottle	98.60	98.10	99.90	100.0	100.0	100.0	100.0	99.80	100.0
Cable	90.30	93.20	100.0	99.50	98.95	100.0	100.0	99.10	98.97
Capsule	76.70	98.60	98.60	98.10	91.90	100.0	99.30	97.10	99.56
Hazelnut	92.00	98.90	93.30	100.0	99.89	100.0	96.80	99.60	99.71
Metal nut	94.00	96.90	86.60	100.0	99.85	100.0	91.90	99.10	100.0
Pill	86.10	96.50	99.80	96.60	97.51	99.40	99.90	98.60	98.80
Screw	81.30	99.50	90.70	98.10	96.43	97.80	99.70	97.60	96.31
Toothbrush	100.0	98.90	97.50	100.0	96.38	94.40	95.20	91.90	91.39
Transistor	91.50	81.00	99.80	100.0	97.83	99.80	99.10	99.30	99.92
Zipper	97.90	98.80	99.90	98.80	98.03	99.50	98.50	99.70	98.74
Av. objects	90.84	96.04	96.61	99.11	97.68	99.09	98.04	98.18	98.34
Av. total	92.07	96.21	97.12	99.06	98.13	99.37	98.27	98.72	98.87

The two best results for each row are in bold. Comparison with best performing and flow-based methods: P.SVDD [27], SPADE [29], Cut-Paste [33], PatchCore [31], PEFM [62], FastFlow [19], CFlow [35], and CS-Flow [20]. Our method achieves state-of-the-art results also for the image-level metric (image-level *AUROC*). Among the flow-based methods, we rank second, only after FastFlow, which uses an unfair test-set tuning

Table 8 Image-level *AUROC* results for LGG MRI [25], ShanghaiTech Campus (STC) [26], and BeanTech (BT) [24] datasets

	LGG MRI	STC	BT01	BT02	BT03	BT AV.
IMAGE						
FastFlow	61.49	74.04	100.0	89.13	96.65	95.26
CFlow	42.56	66.39	94.95	79.68	99.96	91.53
CS-Flow	67.41	98.46	99.81	87.26	99.94	95.67
U-Flow (ours)	72.60	64.59	99.42	88.72	99.72	95.95

The best method for each dataset is in bold. Comparison with best-performing flow-based models: FastFlow [19], CFlow [35], and CS-Flow [20]

Supplementary Information The online version contains supplementary material available at <https://doi.org/10.1007/s10851-024-01193-y>.

Acknowledgements The authors thank Agencia Nacional de Investigación e Innovación, Uruguay, for partially funding this work through a graduate scholarship.

Author Contributions Main author is MT. AP and PM contributed during the whole process. All authors reviewed the manuscript.

Declarations

Conflict of interest The authors declare no conflict of interest.

References

- Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., Bengio, Y.: Generative adversarial nets. In: *Advances in Neural Information Processing Systems*, pp. 2672–2680 (2014)
- Schlegl, T., Seeböck, P., Waldstein, S.M., Schmidt-Erfurth, U., Langs, G.: Unsupervised anomaly detection with generative adversarial networks to guide marker discovery. In: *International Conference on Information Processing in Medical Imaging*, pp. 146–157. Springer (2017)
- Kingma, D.P., Welling, M.: Auto-encoding variational bayes. In: *2nd International Conference on Learning Representations* (2014)
- Yang, J., Shi, Y., Qi, Z.: Dfr: deep feature reconstruction for unsupervised anomaly segmentation. *arXiv preprint arXiv:2012.07122* (2020)
- Dinh, L., Krueger, D., Bengio, Y.: Nice: Non-linear independent components estimation. *arXiv preprint arXiv:1410.8516* (2014)
- Dinh, L., Sohl-Dickstein, J., Bengio, S.: Density estimation using real nvp. *arXiv preprint arXiv:1605.08803* (2016)
- Kingma, D.P., Dhariwal, P.: Glow: Generative flow with invertible 1x1 convolutions. In: *Advances in Neural Information Processing Systems*, vol. 31 (2018)
- Desolneux, A., Moisan, L., Morel, J.-M.: From Gestalt Theory to Image Analysis: A Probabilistic Approach. *Interdisciplinary Applied Mathematics* (2008)
- Zhou, C., Paffenroth, R.C.: Anomaly detection with robust deep autoencoders. In: *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 665–674 (2017)
- Bergmann, P., Löwe, S., Fauser, M., Sattlegger, D., Steger, C.: Improving unsupervised defect segmentation by applying structural similarity to autoencoders. *arXiv preprint arXiv:1807.02011* (2018)
- Gong, D., Liu, L., Le, V., Saha, B., Mansour, M.R., Venkatesh, S., Hengel, A.v.d.: Memorizing normality to detect anomaly: Memory-augmented deep autoencoder for unsupervised anomaly detection. In: *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 1705–1714 (2019)
- Schlegl, T., Seeböck, P., Waldstein, S.M., Langs, G., Schmidt-Erfurth, U.: f-anogan: fast unsupervised anomaly detection with generative adversarial networks. *Med. Image Anal.* **54**, 30–44 (2019)
- Akçay, S., Atapour-Abarghouei, A., Breckon, T.P.: Ganomaly: Semi-supervised anomaly detection via adversarial training. In: *Computer Vision—ACCV 2018: 14th Asian Conference on Computer Vision*, Perth, Australia, December 2–6, 2018, Revised Selected Papers, Part III 14, pp. 622–637 (2019). Springer
- Russakovsky, O., Deng, J., Su, H., Krause, J., Satheesh, S., Ma, S., Huang, Z., Karpathy, A., Khosla, A., Bernstein, M., Berg, A.C., Fei-Fei, L.: ImageNet large scale visual recognition challenge. *Int. J. Comput. Vis. (IJCV)* **115**(3), 211–252 (2015). <https://doi.org/10.1007/s11263-015-0816-y>
- Kim, J.-H., Kim, D.-H., Yi, S., Lee, T.: Semi-orthogonal embedding for efficient unsupervised anomaly segmentation. *arXiv preprint arXiv:2105.14737* (2021)
- Zheng, Y., Wang, X., Deng, R., Bao, T., Zhao, R., Wu, L.: Focus your distribution: Coarse-to-fine non-contrastive learning for anomaly detection and localization. In: *2022 IEEE International Conference on Multimedia and Expo (ICME)*, pp. 1–6 (2022). IEEE
- Lee, S., Lee, S., Song, B.C.: Cfa: Coupled-hypersphere-based feature adaptation for target-oriented anomaly localization. *arXiv preprint arXiv:2206.04325* (2022)
- Yang, J., Shi, Y., Qi, Z.: Dfr: Deep feature reconstruction for unsupervised anomaly segmentation. *arXiv:2012.07122 preprint* (2020)
- Yu, J., Zheng, Y., Wang, X., Li, W., Wu, Y., Zhao, R., Wu, L.: Fastflow: unsupervised anomaly detection and localization via 2d normalizing flows. *arXiv preprint arXiv:2111.07677* (2021)
- Rudolph, M., Wehrbein, T., Rosenhahn, B., Wandt, B.: Fully convolutional cross-scale-flows for image-based defect detection. In: *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, pp. 1088–1097 (2022)
- Touvron, H., Cord, M., Sablayrolles, A., Synnaeve, G., Jégou, H.: Going deeper with image transformers. In: *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 32–42 (2021)
- Akçay, S., Ameln, D., Vaidya, A., Lakshmanan, B., Ahuja, N., Genc, U.: Anomalib: a deep learning library for anomaly detection. In: *2022 IEEE International Conference on Image Processing (ICIP)*, IEEE, pp. 1706–1710 (2022)
- Bergmann, P., Fauser, M., Sattlegger, D., Steger, C.: Mvtec ad—a comprehensive real-world dataset for unsupervised anomaly detection. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 9592–9600 (2019)
- Mishra, P., Verk, R., Fornasier, D., Picciarelli, C., Foresti, G.L.: Vt-adl: A vision transformer network for image anomaly detection and localization. In: *2021 IEEE 30th International Symposium on Industrial Electronics (ISIE)*, pp. 01–06 (2021)
- Buda, M., Saha, A., Mazurowski, M.A.: Association of genomic subtypes of lower-grade gliomas with shape features automatically extracted by a deep learning algorithm. *Comput. Biol. Med.* **109**, 218–225 (2019)
- Liu, W., Luo, D.L., Gao, S.: Future frame prediction for anomaly detection—a new baseline. In: *2018 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (2018)
- Yi, J., Yoon, S.: Patch svdd: Patch-level svdd for anomaly detection and segmentation. In: *Proceedings of the Asian Conference on Computer Vision* (2020)
- Tsai, C.-C., Wu, T.-H., Lai, S.-H.: Multi-scale patch-based representation learning for image anomaly detection and segmentation. In: *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, pp. 3992–4000 (2022)
- Cohen, N., Hoshen, Y.: Sub-image anomaly detection with deep pyramid correspondences. *arXiv preprint arXiv:2005.02357* (2020)
- Defard, T., Setkov, A., Loesch, A., Audigier, R.: Padim: a patch distribution modeling framework for anomaly detection and localization. In: *International Conference on Pattern Recognition*, pp. 475–489 (2021). Springer
- Roth, K., Pemula, L., Zepeda, J., Schölkopf, B., Brox, T., Gehler, P.: Towards total recall in industrial anomaly detection. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 14318–14328 (2022)

32. Yamada, S., Kamiya, S., Hotta, K.: Reconstructed student-teacher and discriminative networks for anomaly detection. arXiv preprint [arXiv:2210.07548](https://arxiv.org/abs/2210.07548) (2022)
33. Li, C.-L., Sohn, K., Yoon, J., Pfister, T.: Cutpaste: Self-supervised learning for anomaly detection and localization. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 9664–9674 (2021)
34. Rudolph, M., Wandt, B., Rosenhahn, B.: Same same but different: Semi-supervised defect detection with normalizing flows. In: Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision, pp. 1907–1916 (2021)
35. Gudovskiy, D., Ishizaka, S., Kozuka, K.: Cflow-ad: Real-time unsupervised anomaly detection with localization via conditional normalizing flows. In: Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision, pp. 98–107 (2022)
36. Simonyan, K., Zisserman, A.: Very deep convolutional networks for large-scale image recognition. arXiv preprint [arXiv:1409.1556](https://arxiv.org/abs/1409.1556) (2014)
37. He, K., Zhang, X., Ren, S., Sun, J.: Deep residual learning for image recognition. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 770–778 (2016)
38. Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., et al.: An image is worth 16x16 words: Transformers for image recognition at scale. arXiv preprint [arXiv:2010.11929](https://arxiv.org/abs/2010.11929) (2020)
39. Fan, H., Xiong, B., Mangalam, K., Li, Y., Yan, Z., Malik, J., Feichtenhofer, C.: Multiscale vision transformers. In: Proceedings of the IEEE/CVF International Conference on Computer Vision, pp. 6824–6835 (2021)
40. Li, Y., Wu, C.-Y., Fan, H., Mangalam, K., Xiong, B., Malik, J., Feichtenhofer, C.: Improved multiscale vision transformers for classification and detection. arXiv preprint [arXiv:2112.01526](https://arxiv.org/abs/2112.01526) (2021)
41. Ronneberger, O., Fischer, P., Brox, T.: U-net: Convolutional networks for biomedical image segmentation. In: International Conference on Medical Image Computing and Computer-assisted Intervention, pp. 234–241. Springer (2015)
42. Rezende, D., Mohamed, S.: Variational inference with normalizing flows. In: International Conference on Machine Learning, pp. 1530–1538 (2015)
43. Jacobsen, J.-H., Smeulders, A., Oyallon, E.: i-revnet: Deep invertible networks. arXiv preprint [arXiv:1802.07088](https://arxiv.org/abs/1802.07088) (2018)
44. Von Gioi, R.G., Jakubowicz, J., Morel, J.-M., Randall, G.: On straight line segment detection. *J. Math. Imaging Vis.* **32**(3), 313–347 (2008)
45. Cao, F., Delon, J., Desolneux, A., Musé, P., Sur, F.: A unified framework for detecting groups and application to shape recognition. *J. Math. Imaging Vis.* **27**, 91–119 (2007)
46. Gardella, M., Musé, P., Morel, J.-M., Colom, M.: Noisesniffer: a fully automatic image forgery detector based on noise analysis. In: 2021 IEEE International Workshop on Biometrics and Forensics (IWBF), pp. 1–6. IEEE (2021)
47. Grosjean, B., Moisan, L.: A-contrario detectability of spots in textured backgrounds. *J. Math. Imaging Vis.* **33**, 313–337 (2009)
48. Davy, A., Ehret, T., Morel, J.-M., Delbracio, M.: Reducing anomaly detection in images to detection in noise. In: 2018 25th IEEE International Conference on Image Processing (ICIP), pp. 1058–1062 (2018). IEEE
49. Tailanian, M., Musé, P., Pardo, Á.: A contrario multi-scale anomaly detection method for industrial quality inspection. arXiv preprint [arXiv:2205.11611](https://arxiv.org/abs/2205.11611) (2022)
50. Ehret, T., Davy, A., Morel, J.-M., Delbracio, M.: Image anomalies: a review and synthesis of detection methods. *J. Math. Imaging Vis.* **61**, 710–743 (2019)
51. Lowe, D.G.: Perceptual Organization and Visual Recognition. Kluwer Academic Publishers, USA (1985)
52. Desolneux, A., Moisan, L., Morel, J.-M.: Edge detection by Helmholtz principle. *J. Math. Imaging Vis.* **14**(3), 271–284 (2001)
53. Cao, F., Musé, P., Sur, F.: Extracting meaningful curves from images. *J. Math. Imaging Vis.* **22**, 159–181 (2005)
54. Monasse, P., Guichard, F.: Fast computation of a contrast-invariant image representation. *IEEE Trans. Image Process.* **9**(5), 860–872 (2000)
55. Musé, P., Sur, F., Cao, F., Gousseau, Y., Morel, J.-M.: An a contrario decision method for shape element recognition. *Int. J. Comput. Vis.* **69**, 295–315 (2006)
56. Ballester, C., Caselles, V., Monasse, P.: The tree of shapes of an image. *ESAIM Control Optim. Calculus Var.* **9**, 1–18 (2003)
57. Xu, Y., Géraud, T., Najman, L.: Context-based energy estimator: application to object segmentation on the tree of shapes. In: 2012 19th IEEE International Conference on Image Processing, pp. 1577–1580 (2012)
58. Serra, J.: Image Analysis and Mathematical Morphology. Academic Press Inc, Cambridge (1983)
59. Chernoff, H.: A measure of asymptotic efficiency for tests of a hypothesis based on the sum of observations. *Ann. Math. Stat.* **15**, 493–507 (1952)
60. Jensen, I., Guttmann, A.J.: Statistics of lattice animals (polyominoes) and polygons. *J. Phys. A Math. Gen.* **33**(29), 257 (2000). <https://doi.org/10.1088/0305-4470/33/29/102>
61. Gioi, R.G., Hessel, C., Dagobert, T., Morel, J.-M., Franchis, C.: Ground visibility in satellite optical time series based on a contrario local image matching. *Image Process. Line* **11**, 212–233 (2021)
62. Wan, Q., Cao, Y., Gao, L., Shen, W., Li, X.: Position encoding enhanced feature mapping for image anomaly detection. In: 2022 IEEE 18th International Conference on Automation Science and Engineering (CASE), pp. 876–881 (2022)
63. Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Kopf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J., Chintala, S.: Pytorch: An imperative style, high-performance deep learning library. In: Advances in Neural Information Processing Systems, vol 32, pp. 8024–8035 (2019)
64. Falcon, W.: The PyTorch Lightning team: PyTorch Lightning (2019). <https://doi.org/10.5281/zenodo.3828935>. <https://github.com/Lightning-AI/lightning>
65. Ardizzone, L., Bungert, T., Draxler, F., Köthe, U., Kruse, J., Schmier, R., Sorrenson, P.: Framework for Easily Invertible Architectures (FrEIA) (2018–2022). <https://github.com/vislearn/FrEIA>
66. Wightman, R.: PyTorch image models. GitHub (2019). <https://doi.org/10.5281/zenodo.4414861>
67. Akiba, T., Sano, S., Yanase, T., Ohta, T., Koyama, M.: Optuna: A next-generation hyperparameter optimization framework. In: Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (2019)
68. Watanabe, S.: Tree-structured parzen estimator: Understanding its algorithm components and their roles for better empirical performance. arXiv preprint [arXiv:2304.11127](https://arxiv.org/abs/2304.11127) (2023)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.



Matías Tailanian received an electrical engineering degree from Universidad de la República, Uruguay (2012), an M.Sc. in mathematics and statistical learning from École Normale Supérieure Paris-Saclay, France (2015), and is currently pursuing a Ph.D. in electrical engineering at Universidad de la República. In 2018 he joined Digital Sense, where he is a Senior R&D Engineer. He has 10+ years of working in the industry in machine vision projects and worked as an Assistant Profes-

sor at Universidad de la República, Uruguay. His research interests include anomaly detection, image processing, and machine learning.



Álvaro Pardo received an electrical engineering degree (1998) and a Ph.D. in electrical engineering (2003), both from Universidad de la República, Uruguay. He is Full Professor at the Department of Engineering of the Catholic University of Uruguay, and was Dean of the School of Engineering and Technologies from 2012–2018. His fields of expertise are Image and Video Restoration, Applications of Pattern Recognition and Data Analysis. He has over 20 years of experience man-

aging R&D projects and almost 20 years of experience in innovation and R&D in industrial projects.



Pablo Musé received an Electrical Engineering degree from the Universidad de la República, Uruguay, in 1999, an M.Sc. degree in mathematics and statistical learning, and a Ph.D. degree in applied mathematics from École Normale Supérieure Paris-Saclay, France, in 2001 and 2004, respectively. Since 2008, he has been with the Division of Electrical Engineering, Universidad de la República, where he is currently a Full Professor of signal processing. He is also an associate researcher at

Centre Borelli of École Normale Supérieure Paris-Saclay, and a co-founder and managing partner at Digital Sense. His research interests include machine learning, image restoration and analysis, computational photography, and remote sensing.